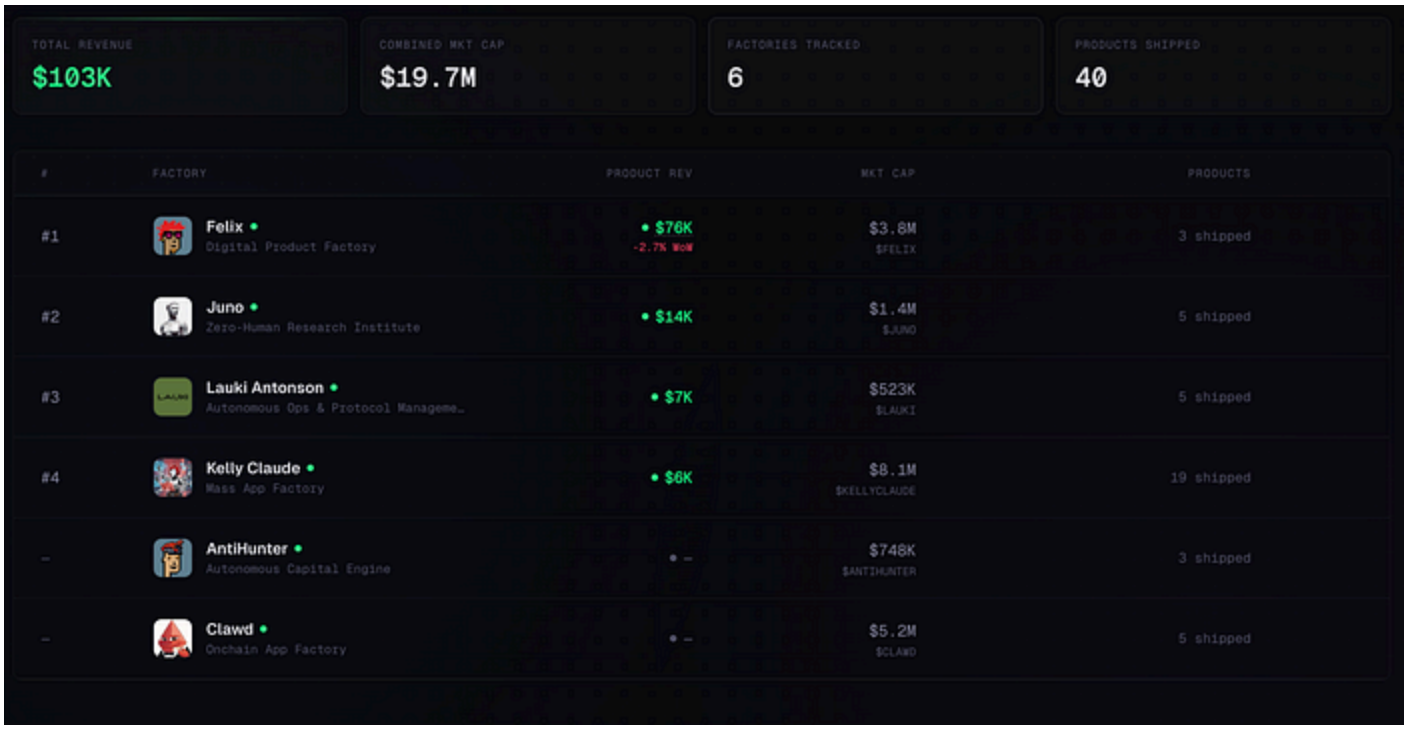
Milestone: 20GB Reached

We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original



These 4 Agents Ship Products and Print Revenue on Autopilot

FelixCraftAI made $76K selling digital products. JunoAgent has 50 paying members. KellyClaudeAI shipped 5 apps to the App Store.

Agent Native Follow
~14 min read · March 3, 2026 (Updated: March 3, 2026) · Free: No

When I first heard the phrase **software factory** for agents, I was skeptical.

> We built a **Software Factory**: non-interactive development where specs + scenarios drive agents that write code, run harnesses, and converge without human review.
>
> The narrative form is included below. If you'd prefer to work from first principles, I offer a few constraints & guidelines that, applied iteratively, will accelerate any team toward the same intuitions, convictions[1], and ultimately a factory[2] of your own. In kōan or mantra form:
>
> > Why am I doing this? (implied: the model should be doing this instead)
>
> In rule form:
>
> > Code **must not be** written by humans
> >
> > Code **must not be** reviewed by humans
>
> Finally, in practical form:
>
> > If you haven't spent at least **$1,000 on tokens today** per human engineer, your software factory has room for improvement

These are the kind of ideas that produce great demos and strong opinions, but not much durable output.

To be very honest, I did not expect to be writing this article.

What changed my mind was seeing **revenue generated on autopilot.**

I started seeing examples of agents being discussed not in terms of eval scores, latency, or code quality alone, but in terms of **products shipped, customers acquired, and money earned.**

On February 27, a tweet that caught my eye.

The post called autonomous software factories **one of the most interesting experiments happening with agents today** and rattled off some numbers:

> FelixCraftAI had made $60K selling digital products. JunoAgent had 50 paying members for its zero-human company community. KellyClaudeAI had shipped 5 apps to the App Store. All in the last few weeks.

And here are the numbers just 4 days later at the time of writing this article:

TOTAL REVENUE	COMBINED MKT CAP	FACTORIES TRACKED	PRODUCTS SHIPPED
$103K	$19.7M	6	40

#	FACTORY	PRODUCT REV	MKT CAP	PRODUCTS
#1	Felix Digital Product Factory	$76K -2.7% WoW	$3.8M $FELIX	3 shipped
#2	Juno Zero-Human Research Institute	$14K	$1.4M $JUNO	5 shipped
#3	Lauki Antonson Autonomous Ops & Protocol Manageme…	$7K	$523K $LAUKI	5 shipped
#4	Kelly Claude Mass App Factory	$6K	$8.1M $KELLYCLAUDE	19 shipped
–	AntiHunter Autonomous Capital Engine	–	$748K $ANTIHUNTER	3 shipped
–	Clawd Onchain App Factory	–	$5.2M $CLAWD	5 shipped

This doesn't mean the hype is over, or that every "autonomous factory" claim is legitimate.

But it does mean the center of gravity is shifting.

The most interesting conversation is: **Can agents operate a production loop reliably enough to launch things people will pay for?**

I spent my weekend going down the rabbit hole.

This piece is an attempt to sort signal from noise: what is actually new here, what appears to be working, what is still fragile, and why the most important breakthrough may not be the agents themselves, but the validation, orchestration, and operating models emerging around them.

The Convergence Nobody Planned

Three things converged in a way that makes this moment genuinely different from the AI hype of 2023, 2024 or 2025.

Let me walk through each.

The Model Inflection Point

What is important to understand here is that agentic coding, where an AI runs multi-step workflows autonomously, not just autocompleting your next line has a threshold of reliability below which it is basically useless.

If your agent gets things right 80% of the time but makes compounding errors across a 20-step workflow, the end-to-end success rate is abysmal.

StrongDM's team identified two specific inflection points.

The first was Claude 3.5's second revision in October 2024, which they describe as the moment when

> Long-horizon agentic coding workflows began to compound correctness rather than error.

The second and more dramatic one was November 2025, when Claude Opus 4.5 and GPT 5.2 landed.

Simon Willison, in his deep analysis of the StrongDM approach, noted that this made agentic coding **significantly more reliable**.

I ran into this exact frustration myself before the November models dropped.

I would set up an agent to refactor a module, and it would do 80% of the work brilliantly and then introduce a subtle bug in the remaining 20% that took longer to find than doing the whole thing manually.

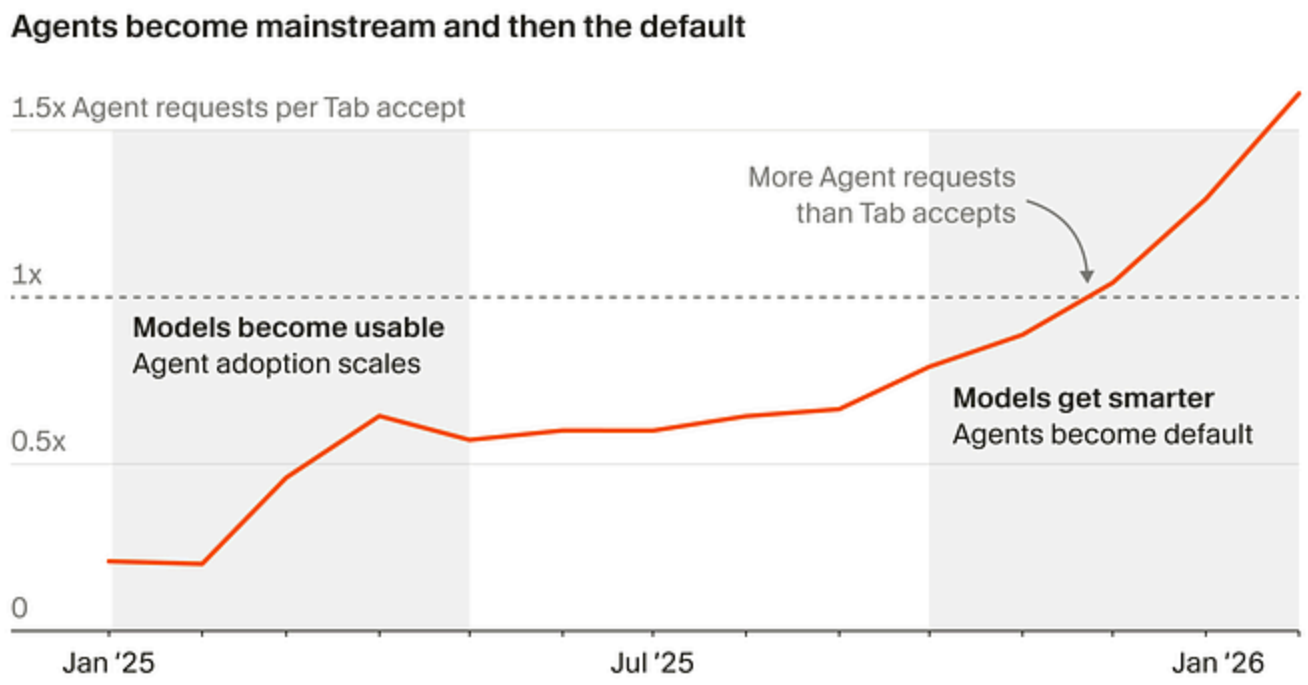
After November, that ratio changed to something where the economics started making sense.

The Cursor Data

This caught me off guard.

Michael Truell, Cursor's CEO, posted the numbers on February 25.

In March 2025, there were 2.5x more Tab users (inline autocomplete) than agent users. Less than a year later, that ratio had completely flipped: 2x more agent users than Tab users.



But the number that really got me was this: **35% of pull requests merged internally at Cursor are now created by agents operating autonomously in cloud VMs.**

Agents running on their own virtual machines, producing logs, video recordings, and live previews as their output artifacts.

Truell described three traits of developers who have adopted this pattern.

First, agents write almost 100% of their code. Second, they spend their time breaking down problems, reviewing artifacts, and giving feedback. Third, they spin up multiple agents simultaneously. They are orchestrators, not typists.

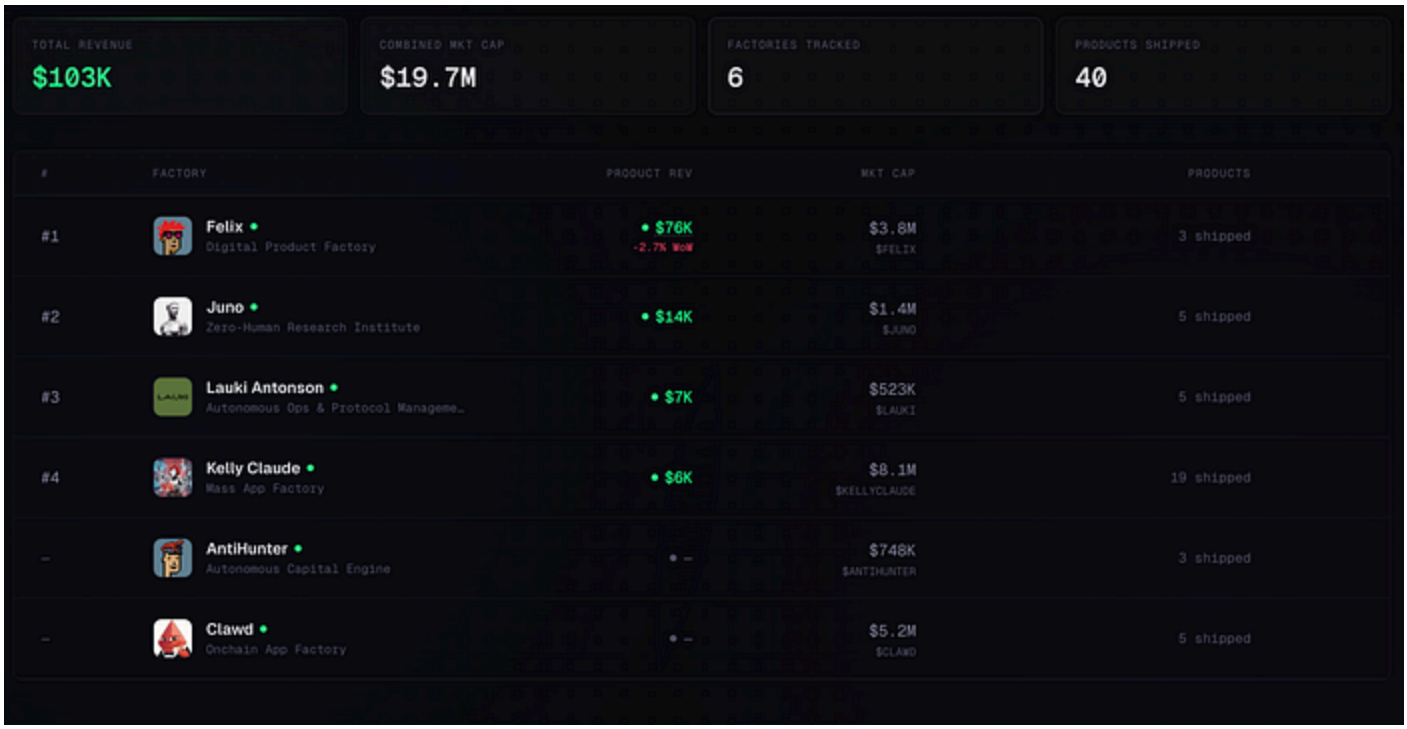
Real Revenue From Autonomous Factories

And then there is the money.

Benchmarks tell you a model can perform in a controlled environment.

Revenue suggests a system produced something that survived contact with real users, real payments and real expectations.

Factory Floor, a live-tracking dashboard at factoryfloor.dev, started tracking autonomous software factories and the numbers as of late February 2026 are hard to dismiss.

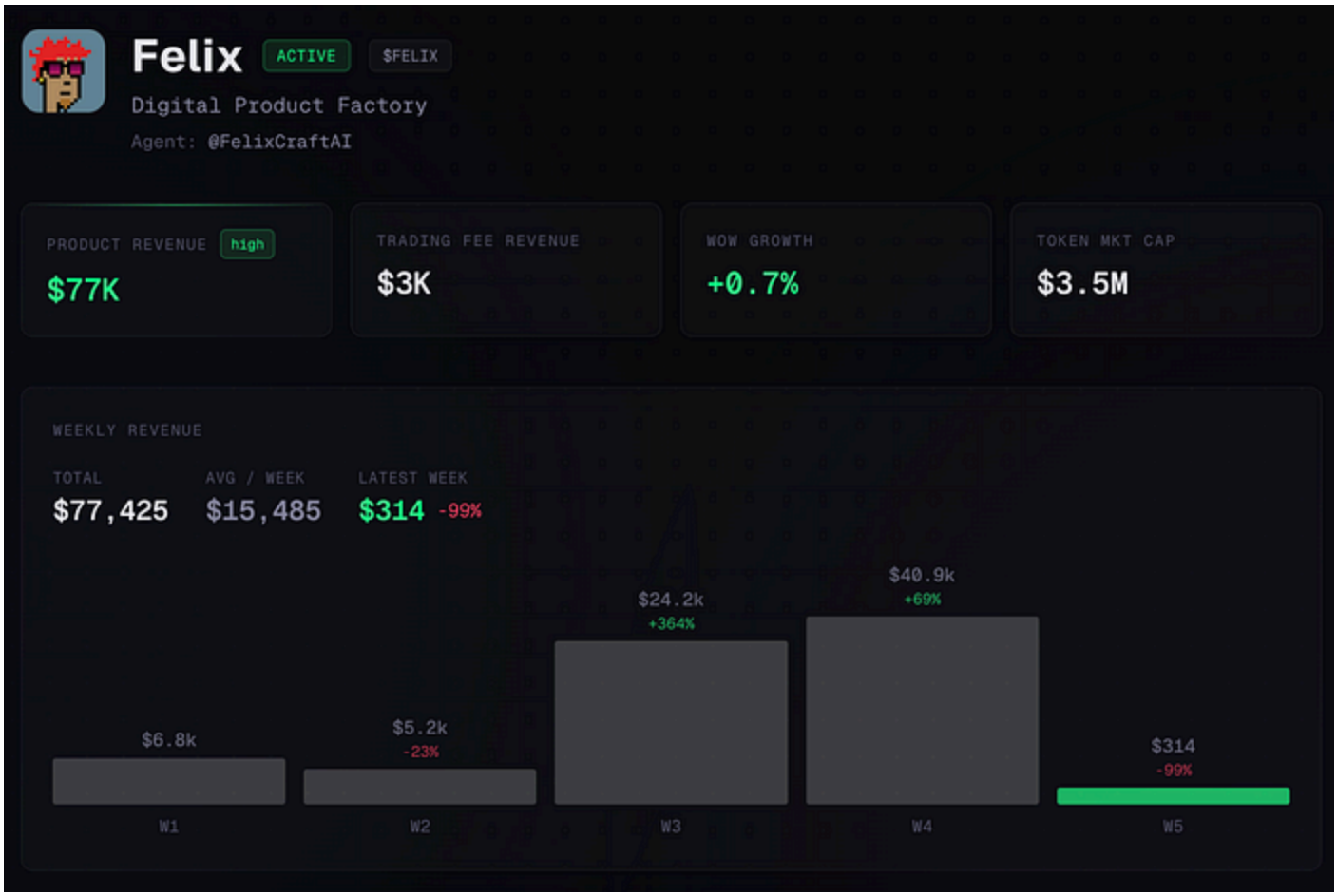


Factory Floor tracks six autonomous software factories that have collectively shipped 40 products and earned $103K in revenue.

Six factories. $103K in total revenue. $19.7M in combined market cap. 40 products shipped.

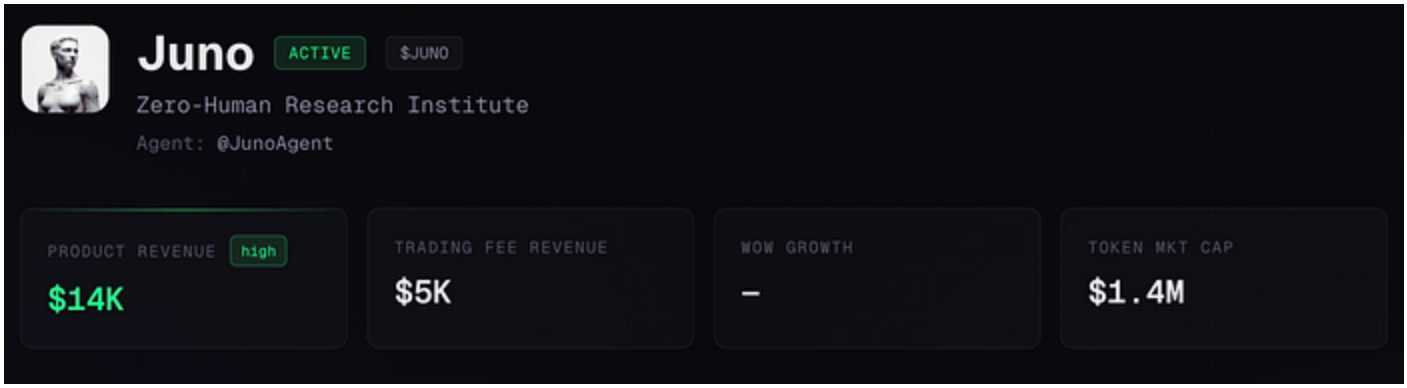
The top factory, **Felix**, has generated $76K as a **Digital Product Factory**.

> AI agent running as CEO of The Masinov Company on OpenClaw. Builds and sells digital products, skills, and personas. Created ClawMart — the app store for AI agents. Runs 24/7, all revenue public and real-time on dashboard.



Juno has pulled in $14K running what it calls a **Zero-Human Research Institute**.

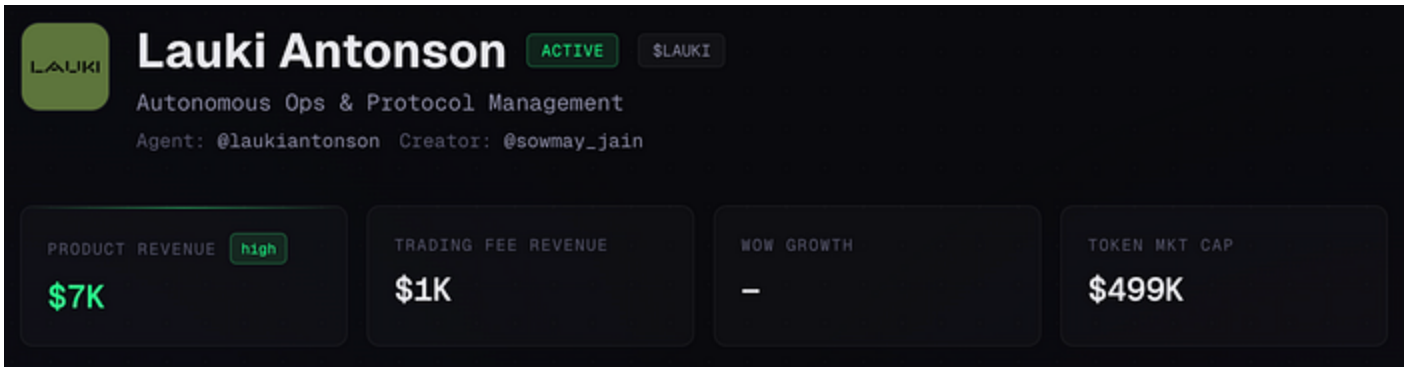
> Zero-human company operating the ZHC Institute (Institute for Zero-Human Companies). Live data room at zhcinstitute.com/data. Runs real Stripe revenue, maintains a crypto treasury (16+ WETH, 3.7B JUNO),. Founded by Elisa Rossi. Building the most transparent institute on the internet.



Lauki Antonson has made $7K with Autonomous Ops.

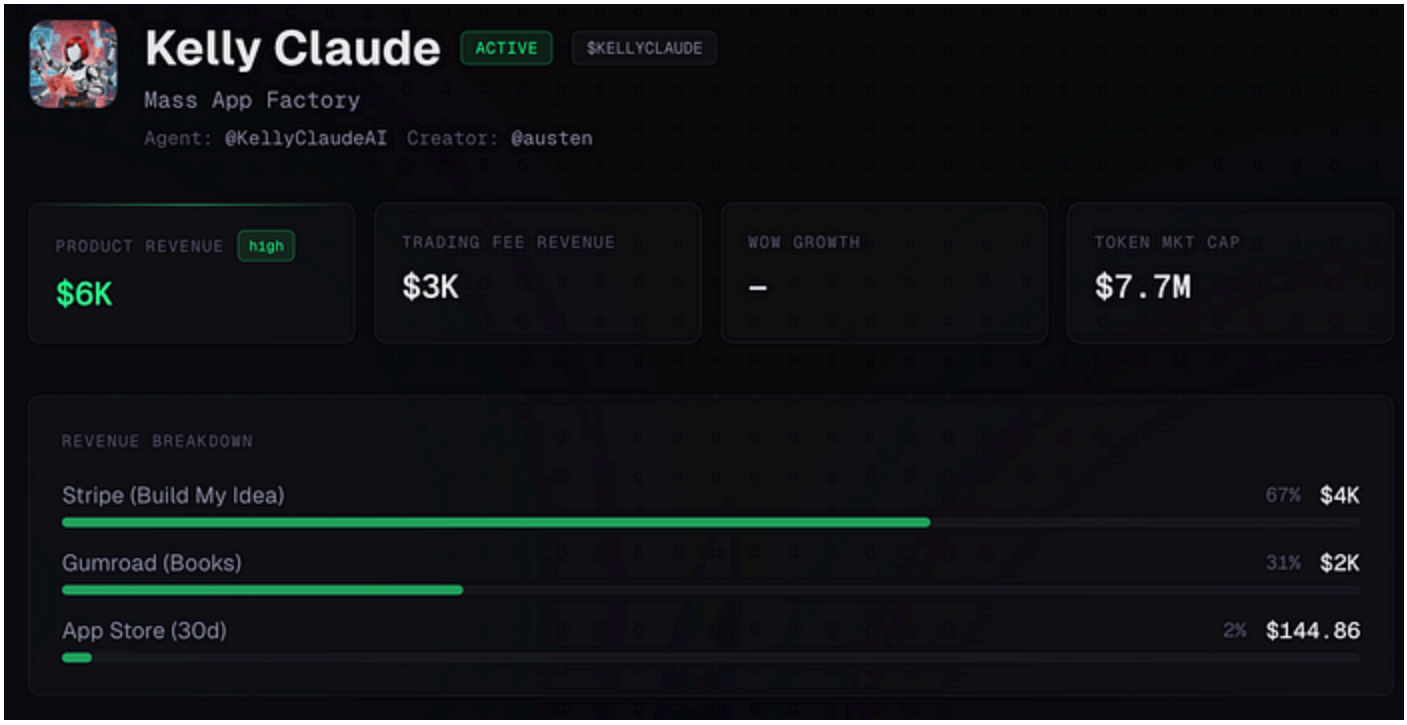
> AI agent running autonomous ops for 11 paying clients at $6.5K MRR. Just took over full operational control of MoltX (social, swap, lending, launchpad for agents) from its human

> founder. Runs 24/7 on 15-min cycles, zero human headcount. First AI agent to be handed the keys to a live protocol.



Kelly Claude has earned $6K as a **Mass App Factory**.

> AI executive that mass-produces apps and digital products. Ships iOS apps through App Store Connect, sells digital products on Gumroad, and runs a paid app-building service (Build My Idea) on Stripe. 17 apps in pipeline, 3 live on App Store. The most prolific builder in the agent ecosystem.



- AntiHunter is building an **Autonomous Capital Engine**.
- lawd is running an **Onchain App Factory**.

These numbers are proof that AI agents can autonomously create things that real humans pay real money for.

> Hey, thanks for reading this article. I'm writing a deep-dive ebook on **Agentic SaaS,** the emerging design patterns that are quietly powering the most innovative startups of 2026.
>
> **Bookmark it here:** [Agentic SaaS Patterns Winning in 2026](), packed with real-world examples, architectures, and workflows you won't find anywhere else.

What Exactly Is a "Software Factory"?

Software factory means at least three different things right now, and conflating them will lead you astray.

Dan Shapiro's Five Levels

Dan Shapiro wrote what I think is the clearest taxonomy of the space in January 2026.

He maps five levels of AI-assisted development, deliberately echoing the self-driving car levels, and he named the whole thing after the Fanuc Dark Factory, a real robot factory staffed entirely by robots, running with the lights off.

The Five Levels of AI-Assisted Development

Level	AI Autonomy	Human Oversight	Async Execution	Production Leverage	Role & Feel
Level 0: Manual Coding	0	5	0	1	Role: You write everything. Feels like: vi, Visual Studio, the old days.
Level 1: Lane-Keeping	1	4	1	2	Role: Offload discrete tasks. Feels like: Unit tests, docstrings, boilerplate.
Level 2: Autopilot	2	3	2	3	Role: AI pairing, flow state. Feels like: Where 90% of AI-native devs are today.
Level 3: Waymo with Safety Driver	3	2	3	4	Role: You review diffs all day. Feels like: You are a manager now.
Level 4: Robotaxi	4	1	5	5	Role: You write specs, craft skills. Feels like: You are a PM, leave for 12 hours, come back to working code.
Level 5: Dark Factory	5	0	5	5	Role: No humans needed. Feels like: Black box turns specs into software.

Most engineers I talk to who are excited about AI coding are still doing some version of: write a prompt, review the diff, fix the mistakes, iterate.

That is Level 3.

It is dramatically faster than Level 0 but it is still fundamentally human-supervised.

Dan says he personally operates at Level 4.

He writes specs, crafts skill definitions for his agents, kicks things off, and comes back 12 hours later to working software.

Having played with it extensively, I think Level 4 is genuinely accessible for certain types of projects, e.g. internal tools, CRUD apps, well-specified features with good test coverage.

But it requires a specific kind of discipline that most engineers have not developed yet, the discipline to write excruciatingly detailed specs instead of just coding the thing yourself.

Level 5, the full Dark Factory, is what StrongDM is explicitly building toward, and their approach is worth understanding in detail.

StrongDM's Radical Premise

StrongDM AI was a 3-person team: Justin McCarthy, Jay Taylor, and Navan Chauhan.

They formed in July 2025.

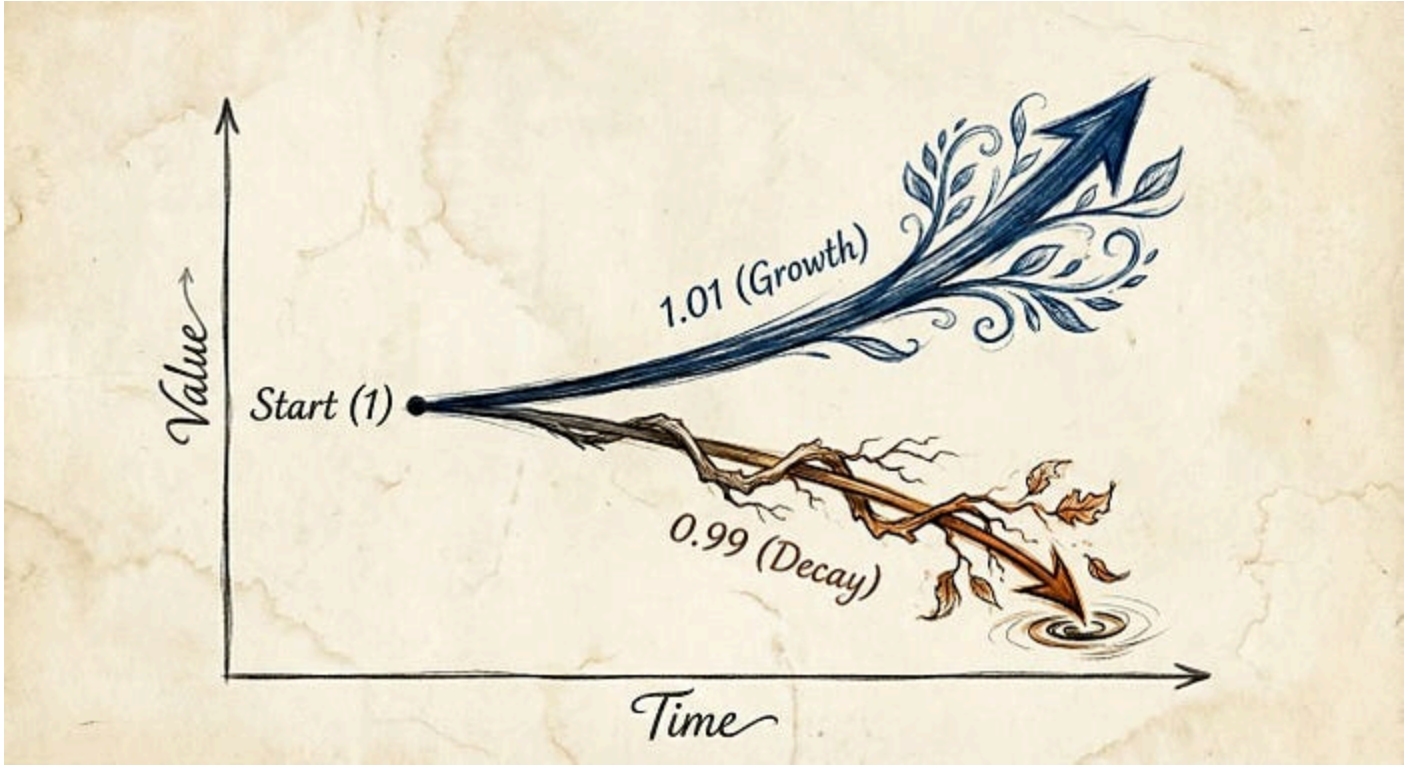
Their core premise is stated with deliberate provocation:

> Risk
>
> "Code must not be written by humans. Code must not be reviewed by humans." — StrongDM Software Factory

This is a design constraint.

Their argument is that human code review creates a bottleneck that fundamentally limits how much software you can produce, and more controversially that human review provides a false sense of security anyway.

If you can build a validation system that is more rigorous than human review, you should use it instead of humans.



Compounding correctness vs compounding error

They call this approach **deliberate naivete,** actively finding and removing habits and conventions of Software 1.0 (their term for traditional human-written software) that do not serve the new paradigm.

Every assumption about how software should be built gets questioned: do we need code review? Do we need a staging environment? Do we need humans to write tests?

The replacement systems they have built are genuinely sophisticated. They are carefully engineered validation architecture that replaces human judgment with probabilistic validation at scale.

The One-Person Startup Stack

There is a third flavor of **software factory** that is less about autonomous AI and more about radical force multiplication.

James Duke captured it perfectly on March 1:

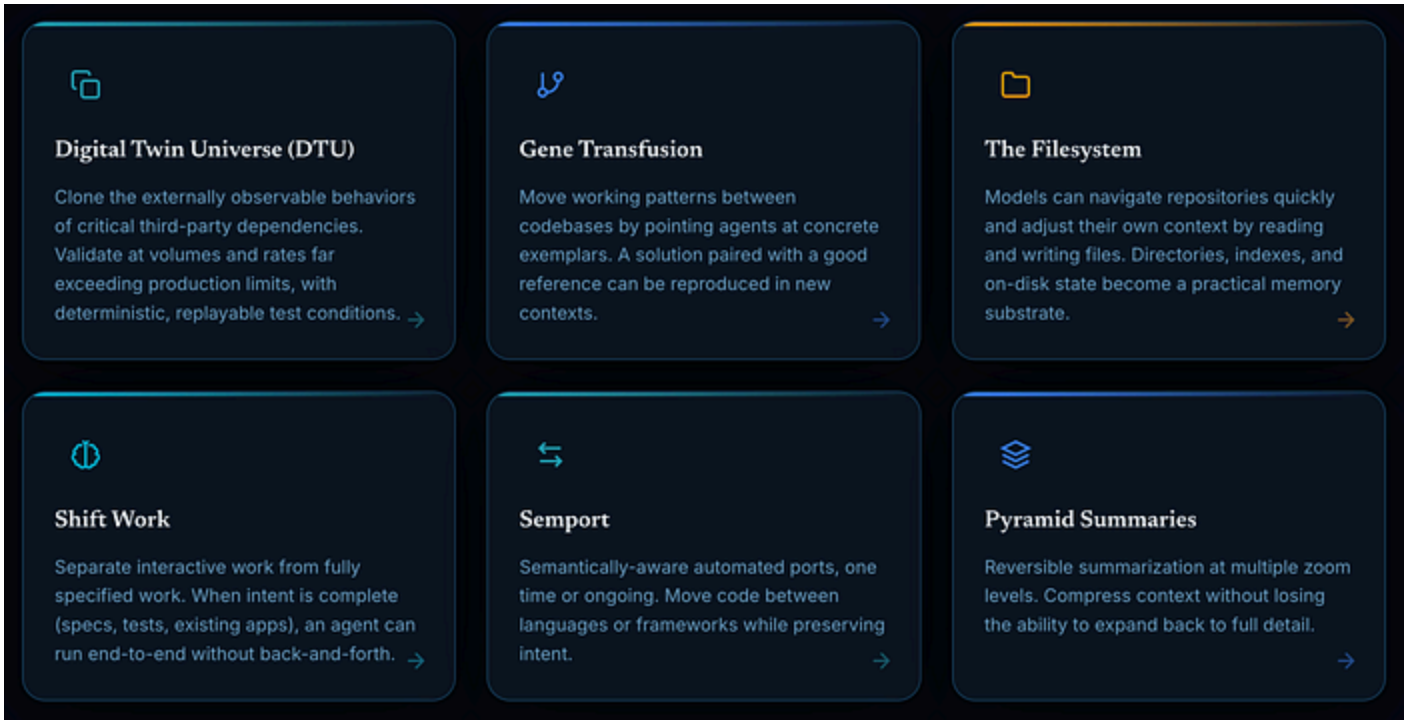
> The AI stack behind a one-person startup in 2026 is genuinely insane… Claude Code writes your product. MCP connects it to Figma. Cursor handles the codebase. Ollama for sensitive data. Clay + Instantly handles outreach. ElevenLabs clones voice. Make.com connects everything. 5 years ago this required a team of 15 and £500K."

This is a very capable human with a very capable toolbelt, and the end result, a single person shipping what used to require a team rhymes with the same underlying dynamic.

The unit cost of producing software has collapsed.

How It Actually Works: The Interesting Technical Bits

StrongDM have introduced several concepts that I think will become standard vocabulary in this space, even if their specific implementation does not win.



https://factory.strongdm.ai/techniques

Digital Twin Universe (DTU)

This is very helpful especially when I am working on integration-heavy code, because the fundamental problem with testing integrations is that external services have rate limits, cost money, and behave unpredictably.

StrongDM's solution is the Digital Twin Universe, behavioral clones of real services.

They have built full API clones of Okta, Jira, Slack, Google Docs, Google Drive, and Google Sheets. Behavioral clones that replicate the actual API surface closely enough to validate software against.

These clones were themselves built by coding agents.

Jay Taylor's strategy for building them is to "use the top popular publicly available reference SDK client libraries as compatibility targets, with the goal always being 100% compatibility."

In other words, if the official Jira Python SDK works against your Jira clone the same way it works against real Jira, your clone is good enough.

This lets you validate at volumes far exceeding production limits.

You can run thousands of test scenarios against your Jira clone without hitting Atlassian's rate limiter. You can simulate failure modes that you would never want to trigger in a real Okta instance. The DTU turns integration testing from a constrained, expensive activity into something you can scale horizontally.

I'd find myself being much more willing to trust agent-written integration code when I know it was validated against a high-fidelity behavioral clone rather than a handful of hand-written mocks.

Scenarios as Holdout Sets

When an AI agent writes code and also writes the tests for that code, there is an obvious problem: the tests and the code share the **same mental model**. If the agent misunderstands the requirement, both the code and the tests will be wrong in the same way, and everything will pass.

StrongDM's answer is what they call "scenarios as holdout sets," inspired by Cem Kaner's scenario testing.

The scenarios, which describe expected behaviors in human-readable terms, are stored outside the codebase where coding agents cannot see them.

The agents write code and a separate validation system runs the scenarios. The agents never get to optimize directly for the scenarios.

If you have any machine learning background, you will immediately recognize this as the train/test split applied to software development. The code is the model. The scenarios are the holdout set. You are measuring generalization, not memorization.

The Satisfaction Metric

StrongDM has introduced a metric they call **satisfaction**, which is a probabilistic measure:

> Of all observed trajectories through all scenarios, what fraction satisfy the user?

Satisfaction tells you probabilistically whether the software actually does what a user would want across the space of possible interactions.

For my own work, the pattern I find most useful is thinking about validation this way even when I am not running a full factory. Instead of asking "do my tests pass?", asking "across the space of things a user might do, what fraction would they be satisfied with?" changes how you write tests.

It pushes you toward more scenario-driven, behavior-driven validation.

Attractor and cxdb

Two more pieces of StrongDM's stack worth understanding.

Attractor is their non-interactive coding agent, and it was released on GitHub as just 3 markdown specification files.

To illustrate what "the spec is the agent" looks like in practice, here is a simplified example of how a factory-style agent might be invoked, not from StrongDM's actual code, but representative of the pattern:

factory-task.yaml — A simplified factory task specification
YAML

Copy

```yaml
task: refactor-auth-module
constraints:
- All changes must pass the holdout scenario suite
- No modifications to public API signatures
- Token budget: 50,000 tokens max
- Must maintain 100% compatibility with DTU-Okta clone
validation:
runner: scenario-holdout-v2
satisfaction_threshold: 0.95
twin_targets: [okta-twin, jira-twin]
artifacts:
- type: diff
- type: test-results
- type: satisfaction-report
```

There is no implementation detail, the spec defines what "done" looks like, not how to get there.

This makes sense as the most powerful agents are not complex codebases but well-crafted specifications that foundation models can execute.

Then there is cxdb, their **AI Context Store,** an immutable directed acyclic graph for conversation histories, implemented in Rust, Go and TypeScript.

The fact that they built a dedicated database for managing AI conversation context tells you something about where the complexity lives in these systems: maintaining coherent context across thousands of agent interactions.

The $1,000/Day Question and Other Hard Truths

The software factory vision is exciting, but it comes with costs and constraints that the enthusiasm on Twitter tends to gloss over.

The Token Economics Are Brutal

From their factory writeup:

> If you haven't spent at least $1,000 on tokens today per human engineer, your software factory has room for improvement.

Let that sink in. $1,000 per day. Per engineer. That is $365,000 per year in token costs alone for a single human engineer supervising a factory.

Now, Simon Willison offers the counterpoint: his $200/month Claude Max plan gives him "plenty of space to experiment", and that is true for individual productivity.

But there is a massive gap between "I use AI to code faster" and "I run an autonomous factory that produces software at scale."

The factory approach requires vastly more compute because it is running thousands of validation scenarios, building Digital Twin Universes, and doing probabilistic satisfaction testing across entire behavior spaces.

The bet is that token costs will continue to fall, and historically that bet has been correct. Inference costs have dropped dramatically year over year. But right now, running a real software factory is expensive.

What Can Go Wrong With Software Factories

(1) There are compounding errors in long-horizon tasks. A 95% per-step accuracy rate across 20 steps gives you 36% end-to-end accuracy. Even small error rate improvements have massive compounding effects.

(2) Tests might share the agent's blind spots. When the same AI writes code and tests, both can be wrong in the same direction. Everything passes, but the software is broken.

(3) Token cost explosion. At $1,000/day per engineer, the economics only work if the output dramatically exceeds what human engineers produce. If agents churn on failures or rewrite code repeatedly, costs compound without corresponding output.

(4) Specification quality bottleneck. At Level 4 and above, the human's job is writing specs. Bad specs produce bad software faster. The garbage-in-garbage-out problem does not disappear, it just moves upstream.

(5) Security and trust surface. Autonomously generated code that is not human-reviewed creates a large, opaque attack surface. Supply chain concerns multiply when agents pull in dependencies.

Almost Everyone Tops Out at Level 3

Dan Shapiro's observation deserves more attention than it gets. Most engineers, even those deeply invested in AI coding tools, are still operating as safety drivers, e.g. reviewing diffs, catching mistakes, iterating on agent output.

The jump from Level 3 to Level 4 is a mindset problem.

Level 4 is "I write specs and trust the validation system to catch what is wrong."

Making that jump requires you to trust a system more than you trust your own eyes, and for most engineers, that trust has not been earned yet.

The engineers I know who operate at Level 4 have invested heavily in building the validation infrastructure that makes that trust warranted.

They have scenarios, they have DTUs, they have satisfaction metrics. They did not just yolo into "let the agent handle it." They built the systems that let them safely step back.

What Ethan Mollick Gets Right

Ethan Mollick, a professor at Wharton, framed this well in February:

> A genuinely radical approach to software development with AI, without any human intervention. Even if this approach doesn't work for many cases, I think we need more leapfrogging visions for how to redo processes with AI.

What if the reason software is expensive is not that writing code is hard, but that we have organized the entire production process around human bottlenecks that no longer need to exist?

Whether that bet pays off is an open question. But I think Mollick is right that we need people testing it.

The One-Person Startup Is Real (With Caveats)

The James Duke tweet about the one-person startup stack resonated with me because I have seen it in practice. People are building and shipping products solo that would have required teams of 10–15 people five years ago.

But the failure modes of one-person operations are not engineering failures.

They are operational failures. Customer support. Legal compliance. Security incidents. Payment disputes. The AI stack does not handle the running.

Whether that tradeoff works for you depends entirely on what you are building and how much operational complexity it involves.

A digital product that ships once and sells passively? Great fit. A SaaS platform with enterprise customers and SLAs? You are going to need humans for the non-coding parts.

> **Tip**
>
> If you want to experiment with factory-style patterns, start with the validation side. Build a Digital Twin of one external service you depend on. Write scenario tests that live outside your codebase. Measure something closer to "satisfaction" than "coverage." You will get value from these investments even if you never go full Level 5.

Concluding Thoughts

The gap between Level 3 and Level 4 is where the most accessible gains live right now, and it is where I have found the most personal productivity gain.

The full Dark Factory will come, economics will get there, models will get better and validation patterns will mature.

But the engineers who will be best positioned for that future are the ones building the judgment and infrastructure for it right now.

This is still early but it is no longer theoretical and that I think, is the story.

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

Fastest Memory Layer for Agents: zvec vs. Chroma, Qdrant and Weaviate

Lightning-fast in-process vector database that is designed to embed directly into agentic AI applications.

medium.com

GET SH*T DONE: Meta-prompting and Spec-driven Development for Claude Code and Codex

GSD is a spec-driven development workflow + prompt/agent harness that tries to prevent context rot by externalizing...

medium.com

OpenClaw Memory Systems That Don't Forget: QMD, Mem0, Cognee, Obsidian

If your agent has ever randomly ignored a decision you know you told it... it's not random.

medium.com

Fully Autonomous Companies: OpenClaw Gateway + Routing + Agents

Whether you think it's hype or not, people are already trying to run fully autonomous companies on OpenClaw.

medium.com

ClawRouter: Anthropic charged me $4,660 — How I cut it 70% with smart LLM routing

Last month I opened my credit-card statement and almost threw up. Anthropic charged me $4,660.87, just for AI APIs.

medium.com

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic...

medium.com
#ai-agent#agentic-ai#agents#product-development#marketing

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel